

# Maths for ML Handbook

## Equations and Formulas used in Machine Learning

### 1 Linear Equation

#### 1.1 Slope-Intercept Equation

##### Slope-Intercept Form

$$y = m \cdot x + c$$

Where,

- $m$  - Slope (angle between straight line and x-axis)
- $c$  - intercept (distance of straight line from the origin)
- $x$  - Independent variable
- $y$  - Dependent variable

##### Slope

Given two points  $(x_1, y_1)$  and  $(x_2, y_2)$ , formula to calculate Slope  $m$ :

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

##### Intercept

Given two points  $(x_1, y_1)$  and  $(x_2, y_2)$ , formula to calculate Intercept  $c$ :

$$c = y_1 - (m \cdot x_1) \quad \text{or} \quad c = y_2 - (m \cdot x_2)$$

#### 1.2 Geometric Relationship b/w lines

##### Parallel Lines

When given two lines  $y = m_1x + c_1$  and  $y = m_2x + c_2$  are parallel:

$$m_1 = m_2$$

##### Perpendicular Lines (Orthogonality)

When given two lines  $y = m_1x + c_1$  and  $y = m_2x + c_2$  are perpendicular:

$$m_1 \times m_2 = -1$$

## 1.3 General Line Equation

### Standard Form

General Line Equation (GLE):

$$w_1x + w_2y + w_0 = 0$$

Rearranging above General Line Equation in Slope-Intercept form:

$$y = \left(-\frac{w_1}{w_2}\right)x_1 + \left(-\frac{w_0}{w_2}\right)$$

Where Slope and Intercept can be derived as,

$$m = -\frac{w_1}{w_2} \quad c = -\frac{w_0}{w_2}$$

### Line Equation (2D Hyperplane)

$$w_1x_1 + w_2x_2 + w_0 = 0$$

### Plane Equation (3D Hyperplane)

$$w_1x_1 + w_2x_2 + w_3x_3 + w_0 = 0$$

### nD Hyperplane Equation

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + w_0 = 0$$

## 2 Vector

### 2.1 Vector Representation

Mathematical notation of a vector  $\vec{x}$  having  $d$  dimension containing real numbers:

$$\mathbf{x} \in \mathbb{R}^d$$

Where  $d$  represents number of elements in the vector.

### Row Vector

$$\vec{x} = [x_1 \ x_2 \ \dots \ x_d]$$

## Column Vector

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

## 2.2 Types of Distances

### 1 Euclidean Distance

Formula to calculate Euclidean Distance between two points  $(x_1, y_1)$  and  $(x_2, y_2)$

$$d = \sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2}$$

### 2 Manhattan Distance

Formula to calculate Manhattan Distance between two points  $(x_1, y_1)$  and  $(x_2, y_2)$

$$d = |y_2 - y_1| + |x_2 - x_1|$$

## 2.3 Types of Norms

### L2 Norm of a Vector

Formula for L2 Norm or Euclidean Distance of a vector of  $d$  dimension:

$$\begin{aligned} \|\vec{x}\|_2 &= \sqrt{x_1^2 + x_2^2 + x_3^2 + \dots + x_d^2} \\ &= \sqrt{\sum_{i=1}^d x_i^2} \end{aligned}$$

### L1 Norm of a Vector

Formula for L1 Norm or Manhattan Distance of a vector of  $d$  dimension:

$$\begin{aligned} \|\vec{x}\|_1 &= |x_1| + |x_2| + |x_3| + \dots + |x_d| \\ &= \sum_{i=1}^d |x_i| \end{aligned}$$

## 3 Matrix Multiplication

### 3.1 Matrix Multiplication rule

Given two matrices  $A$  and  $B$  where:

- $A : m \times n$
- $B : n \times k$

#### Rule

Number of columns in the first matrix should be equal to the number of rows in the second matrix.

$$m \times n \Leftrightarrow n \times k = m \times k$$

The resulting matrix will be of shape:  $m \times k$

### 3.2 Matrix Multiplication in NumPy

Given two NumPy matrices `m1` and `m2`

```
m1 @ m2  
# or  
np.matmul(m1, m2)
```

## 4 Dot Product

### 4.1 Dot Product of Vectors

Formula to calculate dot product of two vectors  $\vec{a}$  and  $\vec{b}$ :

$$\vec{a} \cdot \vec{b} = a^T \cdot b$$

$$= \sum_{i=1}^d a_i b_i$$

### 4.2 Dot Product in NumPy

Given two NumPy vectors `v1` and `v2`

```
np.dot(v1, v2)
```

## 4.3 Application of Dot Product

### 1 Angle between Vectors

Formula to calculate angle between two vectors  $\vec{a}$  and  $\vec{b}$

$$\cos \theta = \frac{\vec{a}^T \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

### 2 Machine Learning

In ML context, the dot product is used to represent the below "best fit" line or hyper-plane equation:

$$w_1x_1 + w_2x_2 + \dots + w_dx_d + w_0 = 0$$

using  $\vec{w}$  and  $\vec{x}$  vectors of  $d$  dimensions:

$$\vec{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \quad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

in **Linear Algebraic** form as,

$$\vec{w}^T \cdot \vec{x} + w_0 = 0$$

Where  $\vec{w}^T \cdot \vec{x}$  is the dot product between weight vector  $\vec{w}$  and feature vector  $\vec{x}$ .

### 3 Projection of a Vector

$$\text{Projection of } \vec{x} \text{ on } \vec{y} = \vec{x} \cdot \hat{y}$$

#### Unit Vector

Formula to calculate unit vector  $\hat{w}$  of a  $\vec{w}$  vector:

$$\hat{w} = \frac{\vec{w}}{\|\vec{w}\|}$$

Formula to calculate projection of  $\vec{x}$  on  $\vec{y}$ :

$$\text{Projection of } \vec{x} \text{ on } \vec{y} = \frac{\vec{x} \cdot \vec{y}}{\|\vec{y}\|}$$

## 5 Shifting Lines

### 5.1 One Direction

Given line equation:

$$w_1x_1 + w_2x_2 + w_0 = 0$$

Shifting lines in one direction (x-axis or y-axis) by  $a$  units:

|        |                     |                |   |                |   |           |
|--------|---------------------|----------------|---|----------------|---|-----------|
| x-axis | Right $\rightarrow$ | $w_1(x_1 - a)$ | + | $w_2x_2$       | + | $w_0 = 0$ |
| x-axis | Left $\leftarrow$   | $w_1(x_1 + a)$ | + | $w_2x_2$       | + | $w_0 = 0$ |
| y-axis | Up $\uparrow$       | $w_1x_1$       | + | $w_2(x_2 - a)$ | + | $w_0 = 0$ |
| y-axis | Down $\downarrow$   | $w_1x_1$       | + | $w_2(x_2 + a)$ | + | $w_0 = 0$ |

### 5.2 Two Directions

Shifting lines in two direction (x-axis and y-axis) simultaneously by  $a$  units:

$$w_1(x_1 - a) + w_2(x_2 - a) + w_0 = 0$$

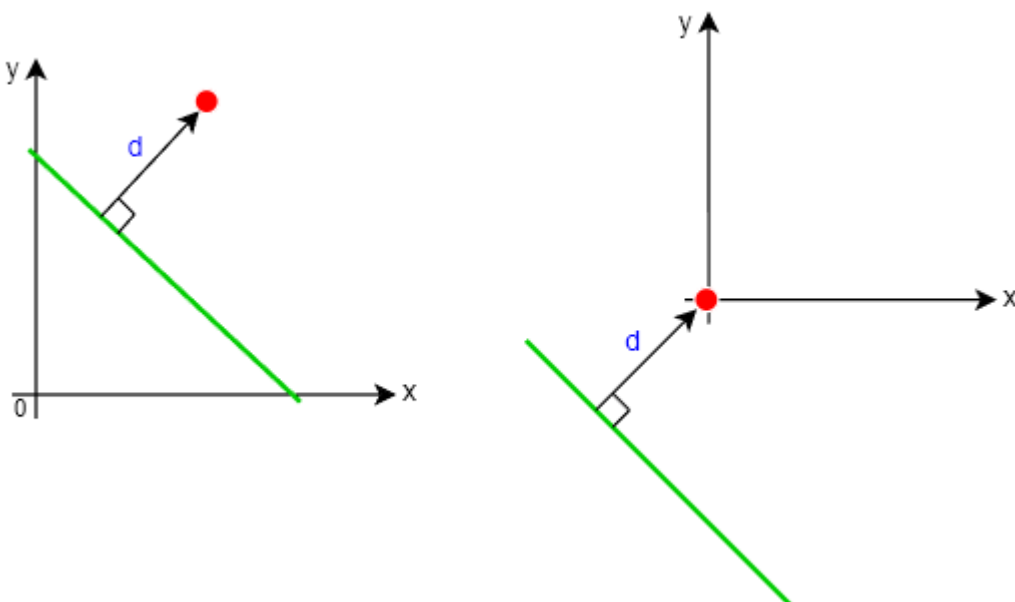
Solving above equation gives:

$$w_1x_1 + w_2x_2 + w'_0 = 0$$

Where  $w'_0$  is:

$$w'_0 = w_0 - a(w_1 + w_2)$$

### 5.3 Usecase



## 6 Distances

### 6.1 Distance Between Line and Origin

Formula to calculate the distance between line (Decision Boundary) and origin using line's Normal vector  $\vec{w}$

$$d = \frac{-w_0}{\|w\|}$$

### 6.2 Distance Between Point and Line

Formula to calculate the distance between a point  $x_0$  and line (Decision Boundary) using line's Normal vector  $\vec{w}$

$$d = \frac{w^T x_0 + w_0}{\|w\|}$$

### 6.3 Distance Between Parallel Lines

#### Same Normal Vector

Formula to calculate the distance between two parallel lines with Normal vector  $w$  having same magnitude:

$$d = \frac{w_0^{(1)} - w_0^{(2)}}{\|w\|}$$

#### Different Normal Vector

Formula to calculate the distance between two parallel lines with Normal vectors of different magnitudes where  $w^{(1)} = \lambda \cdot w^{(2)}$

$$d = \frac{w_0^{(1)}}{\|w^{(1)}\|} - \frac{w_0^{(2)}}{\|w^{(2)}\|}$$

# 7 Binary Classification

## 7.1 Half-spaces

### Positive Half-space

In Positive Half-space all data-points have **distance greater than zero** because,

$$w_1x_1 + w_2x_2 + w_0 > 0$$

### Negative Half-space

In Negative Half-space all data-points have **distance less than zero** because,

$$w_1x_1 + w_2x_2 + w_0 < 0$$

### Decision Boundary

On Decision Boundary all data-points have **distance equal to zero** because,

$$w_1x_1 + w_2x_2 + w_0 = 0$$

## 7.2 Mathematical Notation

### Dataset

Equation to represent a labeled dataset  $D$  used in Binary Classification:

$$D := \left\{ (x_i, y_i) ; x \in \mathbb{R}^d ; y \in \{-1, +1\} \right\}_{i=1}^n$$

Where,

- $D$  is Dataset
- $x$  are features
- $y$  are labels
- $n$  is the size of the dataset

### Label

Actual Label  $y$  can take values:

$$y \begin{cases} +1 & \text{in +ve Half-Space} \\ -1 & \text{in -ve Half-Space} \end{cases}$$



## Classifier

Mapping function for Binary Classification: nD Hyperplane

$$\hat{y} = f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + w_0$$

or

$$= w^T x + w_0$$

Predicted label  $\hat{y}$  can take values:

$$\hat{y} \begin{cases} +1 & \text{if } (w^T x + w_0) \geq 0 \\ -1 & \text{if } (w^T x + w_0) < 0 \end{cases}$$

## 7.3 Gain Function

Equation of Gain Function for Binary Classification:

$$g(D, \vec{w}, w_0) = \frac{1}{n} \sum_{i=1}^n \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) \cdot y^{(i)}$$

## 7.4 Loss Function

Equation of Loss Function for Binary Classification:

Loss Function = – Gain Function

$$l(D, \vec{w}, w_0) = -\frac{1}{n} \sum_{i=1}^n \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) \cdot y^{(i)}$$

## 7.5 Perceptron Learning Algorithm

### Weight Updation Logic

Logic in Perceptron Learning Algorithm to update Weights for miss-classified data-point:

$$\text{if } \text{sign} \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) \neq \text{sign}(y^{(i)}) :$$

$$w = w + x^{(i)} \cdot y^{(i)}$$

$$w_0 = w_0 + y^{(i)}$$

## 8 Optimization

### 8.1 Optimization of Gain Function

Equation for Optimization of Gain Function for Binary Classification:

$$\begin{aligned}w^*, w_0^* &= \arg \max_{w, w_0} g(D, \vec{w}, w_0) \\&= \arg \max_{w, w_0} \frac{1}{n} \sum_{i=1}^n \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) \cdot y^{(i)}\end{aligned}$$

**Goal**

**Maximize Gain function.**

### 8.2 Optimization of Loss Function

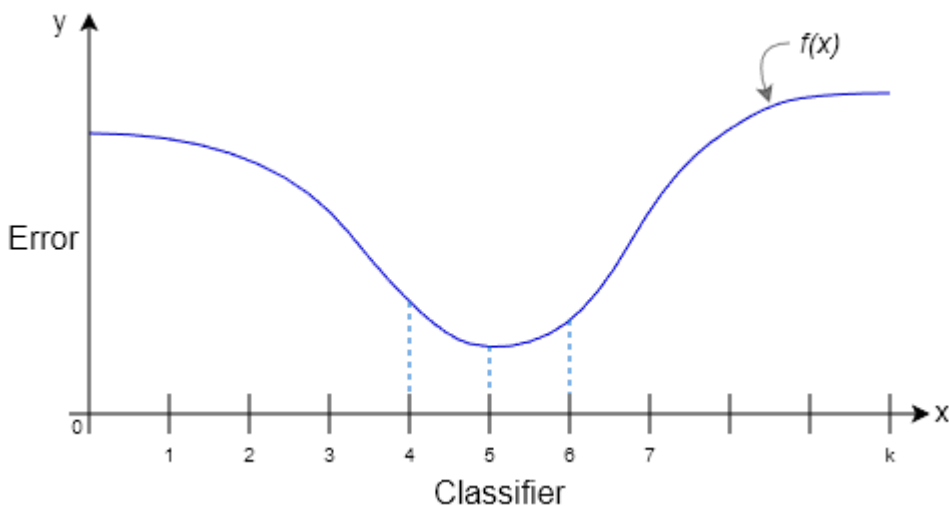
Equation for Optimization of Loss Function for Binary Classification:

$$\begin{aligned}w^*, w_0^* &= \arg \min_{w, w_0} l(D, \vec{w}, w_0) \\&= \arg \min_{w, w_0} -\frac{1}{n} \sum_{i=1}^n \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) \cdot y^{(i)}\end{aligned}$$

**Goal**

**Minimize Loss function.**

**ML Context**



Error vs Classifier is represented as a function  $f(x)$  which needs to be minimized.

# 9 Functions

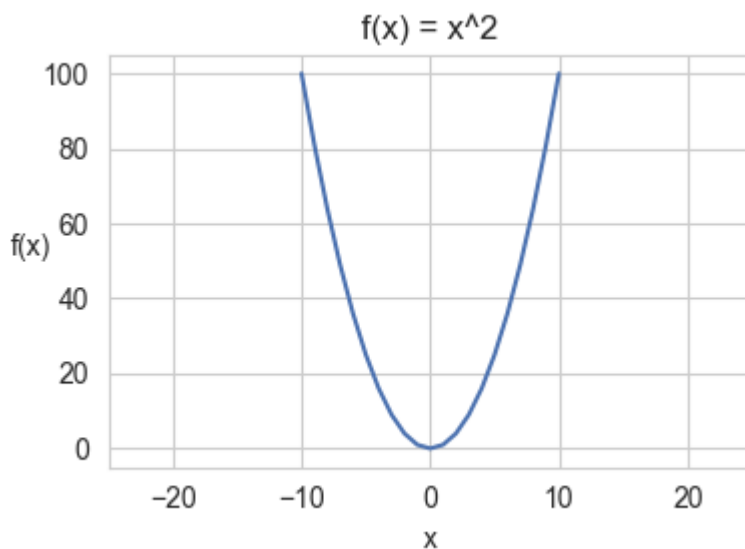
## 9.1 Continuous Function

### Definition

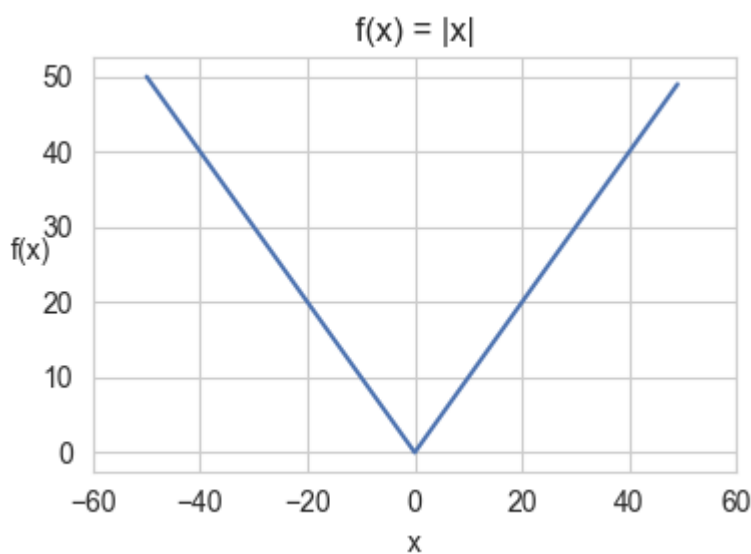
Criteria for a Continuous function:

$$\lim_{x \rightarrow a^-} f(x) = \lim_{x \rightarrow a^+} f(x) = f(a)$$

### Example #1



### Example #2



#### Note:

Even though  $f(x) = |x|$  is continuous function, it is **not differentiable at**  $x = 0$

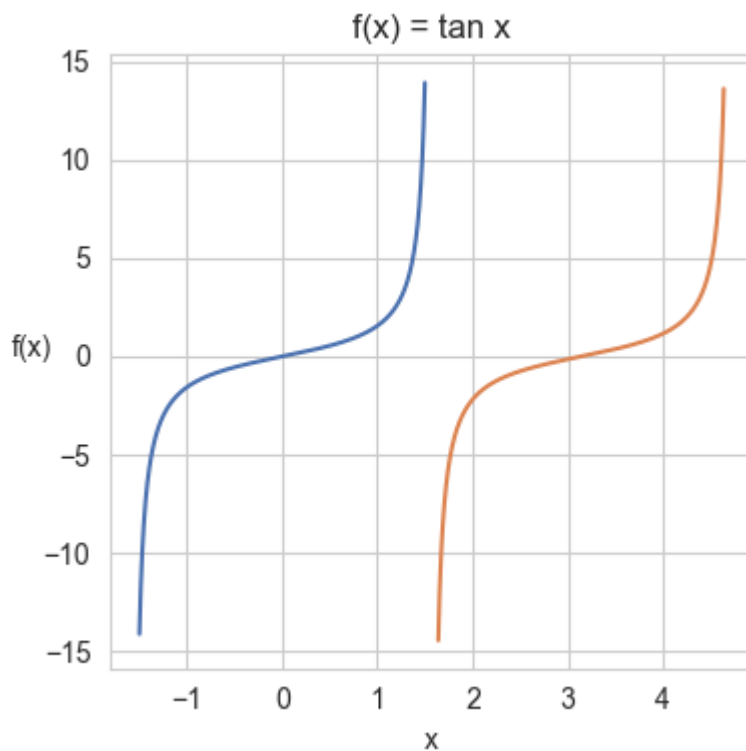
## 9.2 Discontinuous Function

### Definition

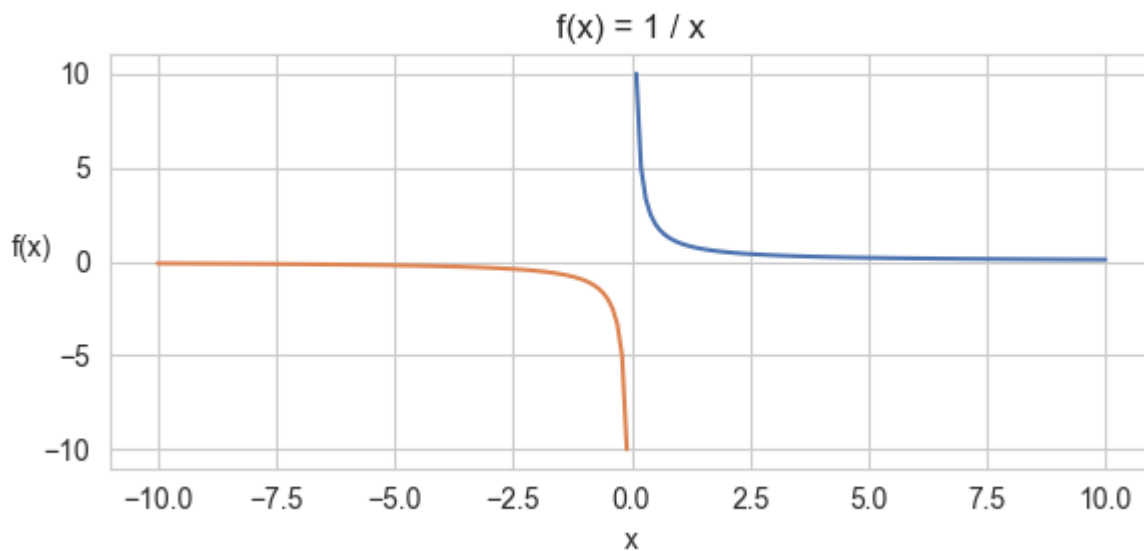
Criteria for a Discontinuous function:

$$\lim_{x \rightarrow a^-} f(x) \neq \lim_{x \rightarrow a^+} f(x)$$

### Example #1



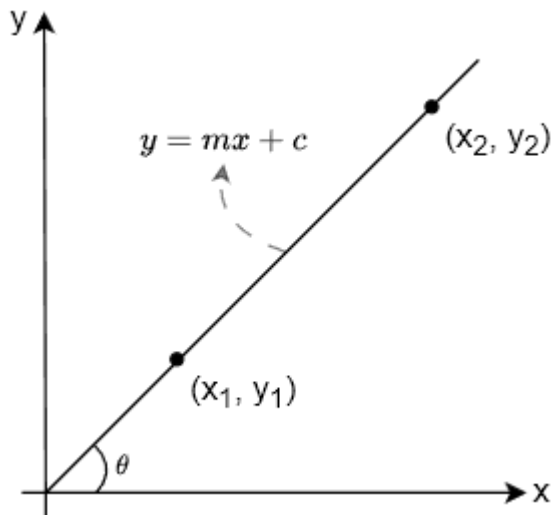
### Example #2



# 10 Differentiation

## 10.1 Secant

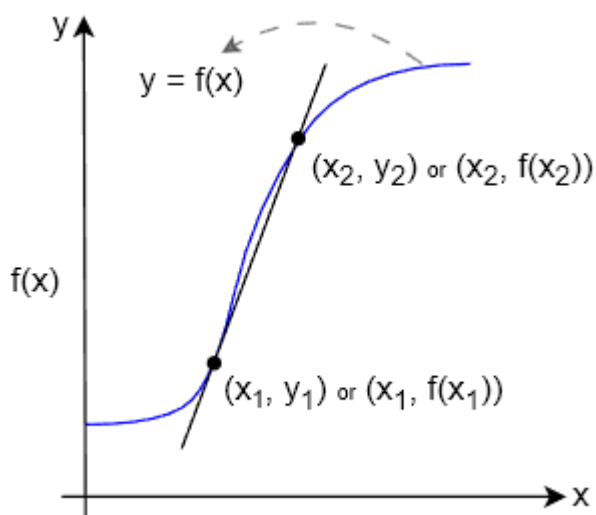
### Slope of Line



Formula to calculate slope of a line intersecting two points  $(x_1, y_1)$  and  $(x_2, y_2)$

$$m = \tan \theta = \frac{y_2 - y_1}{x_2 - x_1}$$

### Slope of Secant Line

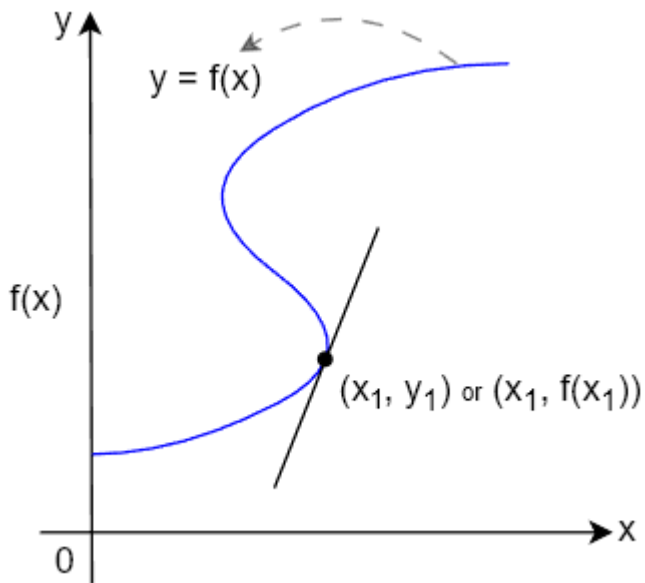


Formula for slope of a secant line intersecting a curve at two points  $(x_1, y_1)$  and  $(x_2, y_2)$

$$m = \frac{f(x_2) - f(x_1)}{x_2 - x_1}$$

## 10.2 Tangent

### Slope of Tangent Line (Derivative)



Formula for slope of a tangent line intersecting a curve at one single point  $(x, y)$

$$f'(x) = \frac{d}{dx} f(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

## 10.3 Derivatives Examples

### Commonly used Derivatives

$$(1) \quad \frac{d}{dx} x^n = nx^{n-1}$$

$$(5) \quad \frac{d}{dx} \sin(x) = \cos(x)$$

$$(2) \quad \frac{d}{dx} \log(x) = \frac{1}{x}$$

$$(6) \quad \frac{d}{dx} \cos(x) = -\sin(x)$$

$$(3) \quad \frac{d}{dx} e^x = e^x$$

$$(7) \quad \frac{d}{dx} \tan(x) = \sec^2(x)$$

$$(4) \quad \frac{d}{dx} c = 0$$

$$(8) \quad \frac{d}{dx} b^x = b^x \log(b)$$

## 10.4 Rules of Differentiation

### 1 Constant Rule

$$\frac{d}{dx} [c \cdot f(x)] = c \cdot \frac{d}{dx} f(x)$$

### 2 Sum Rule

$$\frac{d}{dx} [f(x) + g(x)] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

### 3 Difference Rule

$$\frac{d}{dx} [f(x) - g(x)] = \frac{d}{dx} f(x) - \frac{d}{dx} g(x)$$

### 4 Product Rule

$$\frac{d}{dx} [f(x) \cdot g(x)] = g(x) \frac{d}{dx} f(x) + f(x) \frac{d}{dx} g(x)$$

### 5 Quotient Rule

$$\frac{d}{dx} \left[ \frac{f(x)}{g(x)} \right] = \frac{g(x) \frac{d}{dx} f(x) - f(x) \frac{d}{dx} g(x)}{[g(x)]^2}$$

### 6 Chain Rule

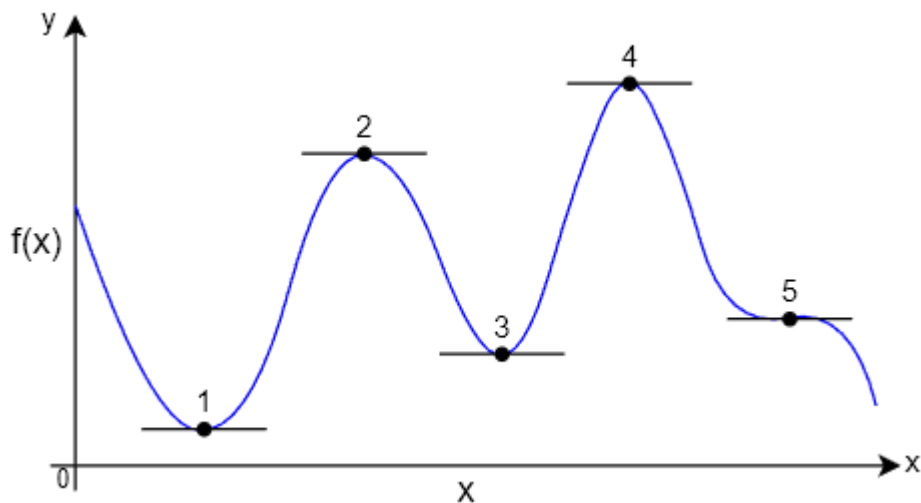
$$\frac{d}{dx} f(g(x)) = \frac{d}{dx} f(g(x)) \cdot \frac{d}{dx} g(x)$$

How Chain rule works?

$$\begin{aligned} \frac{d}{dx} \log(x^2) &= \frac{1}{x^2} \cdot 2x \\ &= \frac{2}{x} \end{aligned}$$

- Keep the inside  $x^2$  as it is and take derivative of outside log i.e.,  $\frac{1}{x^2}$ .
- Multiply it by the derivative of the inside  $x^2$  i.e.,  $2x$ .

## 10.5 Critical Points



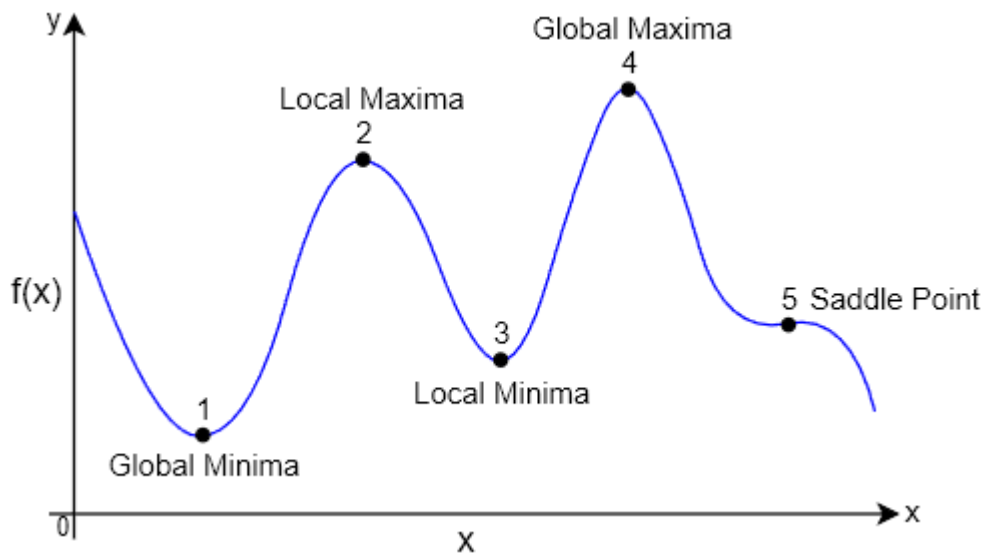
### Definition

Slope of the Tangent line to a curve (i.e., derivative of a function) will be **zero at Critical Points**.

$$f'(x) = 0$$

## 10.6 Local Optima

### Types of Local Optima/Extrema

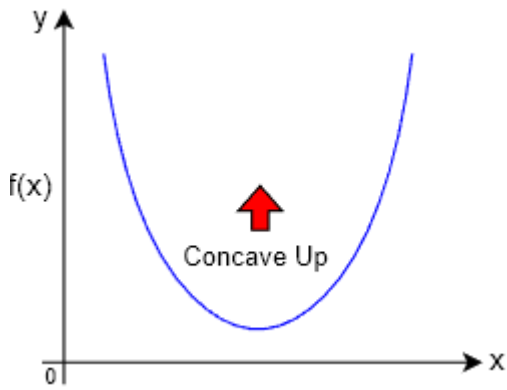


Two types of Local Optima/Extrema where  $f'(x)$  (First derivative) is equal to zero and  $f''(x)$  (Second Derivative) decides the type of Extrema:

$$f'(x) = 0, \quad f''(x) \begin{cases} f''(x) > 0 & \text{Local Minima} \\ f''(x) < 0 & \text{Local Maxima} \end{cases}$$

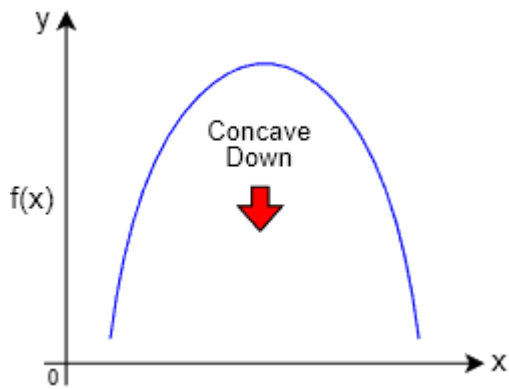


## 1 Local Minima



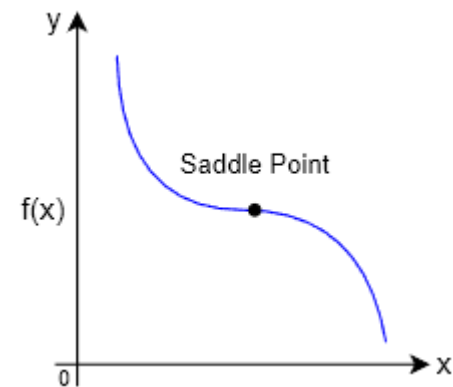
$$f''(x) > 0$$

## 2 Local Maxima



$$f''(x) < 0$$

## Saddle Point



Non-optimal points where both  $f'(x) = 0$  (First derivative) and  $f''(x) = 0$  (Second derivative).

$$f''(x) = 0$$

## 10.7 Principles of Maxima and Minima

### First and Second Derivative Test for Extrema

#### STEP 1: Compute First Derivative

Given function  $f(x)$  compute its first derivative:

$$f'(x)$$

#### STEP 2: Find Critical Point

Equate the first derivative to zero to find the critical point:

$$f'(x) = 0$$

$$x = c$$

Critical Point :- Critical value of  $x$ , say  $c$ , at which slope  $f'(x) = 0$ .

#### STEP 3: Classify Using Second Derivative

To classify the Critical point from STEP 2 to the correct type of local Optima, compute derivative of  $f'(x)$  i.e., second derivative of  $f(x)$  from STEP 1:

$$f''(x)$$

Identify the type of Local Optima based on below rule:

$$f''(x) \begin{cases} \text{if } f''(x) > 0 : & \text{Local Minima} \\ \text{if } f''(x) < 0 : & \text{Local Maxima} \\ \text{if } f''(x) = 0 : & \text{Test Inconclusive} \end{cases}$$

#### STEP 4: Calculate Local Optima Value

Substitute the Critical Value  $c$  from STEP 2 in original function  $f(x)$  to identify the actual value of Local Optima.

$$f(c) \in \mathbb{R}$$

# 11 Partial Differentiation

## 11.1 Partial Derivative

### Partial Derivative of a general function

Formula to calculate Partial derivative of any general function  $f(x, y)$  with two variables  $\{x, y\}$

$$\frac{\partial}{\partial x} f(x, y) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x}$$

$$\frac{\partial}{\partial y} f(x, y) = \lim_{\Delta y \rightarrow 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y}$$

## 11.2 Slope of Tangent Line (Gradient)

### Gradient of a function

Given Binary Classifier having feature vector  $\vec{x}$  of  $d$  dimension and weight vector  $\vec{w}$ :

$$f(w_1, w_2, \dots, w_d, w_0) = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + w_0$$

$$f(\vec{w}, w_0) = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + w_0$$

Equation to calculate Gradient Vector for the above scalar function  $f(\vec{w}, w_0)$

$$\nabla_{\vec{w}, w_0} f(\vec{w}, w_0) = \begin{bmatrix} \frac{\partial}{\partial w_1} f(\vec{w}, w_0) \\ \frac{\partial}{\partial w_2} f(\vec{w}, w_0) \\ \vdots \\ \frac{\partial}{\partial w_d} f(\vec{w}, w_0) \\ \frac{\partial}{\partial w_0} f(\vec{w}, w_0) \end{bmatrix}$$

Visualize Gradient: <https://www.desmos.com/3d/e3fratrwho>

# 12 Gradient Descent

## 12.1 Gradient Descent Equation

### Gradient Update Rule

$$x^n = x^o - \eta \frac{\partial}{\partial x} f(x^o)$$

Where,

- $x$  - Some random vector (not features)
- $x^n$  - New  $x$  values after update
- $x^o$  - Old  $x$  values before update
- $\eta$  - Learning Rate

### Gradient Update Rule (In ML Context)

$$\vec{w}^{\text{new}} = \vec{w}^{\text{old}} - \eta \frac{\partial}{\partial \vec{w}} f(\vec{w}^{\text{old}})$$

$$w_0^{\text{new}} = w_0^{\text{old}} - \eta \frac{\partial}{\partial w_0} f(w_0^{\text{old}})$$

Where,

- $\vec{w}^{\text{new}}$  - New Gradient vector  $w$  after update
- $\vec{w}^{\text{old}}$  - Old Gradient vector  $w$  before update
- $w_0^{\text{new}}$  - New Bias  $w_0$  value after update
- $w_0^{\text{old}}$  - Old Bias  $w_0$  value before update
- $\eta$  - Learning Rate

## 12.2 Constrained Optimization

### Constraint

Equation for Optimization/Minimization of Loss Function with a constraint that  $\vec{w}$  is a unit vector:

$$w^*, w_0^* = \arg \min_{\vec{w}, w_0} l(D, \vec{w}, w_0) \quad \text{such that : } \|\vec{w}\| = 1$$

$$= \arg \min_{\vec{w}, w_0} -\frac{1}{n} \sum_{i=1}^n \left( \frac{w^T x^{(i)} + w_0}{\|w\|} \right) y^{(i)} \quad \text{such that : } \|\vec{w}\| = 1$$

### Lagrangian Function

$$\mathcal{L}(x, \lambda) \equiv f(x) + \lambda \cdot g(x)$$

Where  $\lambda$  is called as **Lagrange Multiplier**.

### Lagrange Multiplier for Constraint Optimization

Applying Lagrange Multiplier on Optimization equation:

$$w^*, w_0^* = \arg \min_{\vec{w}, w_0, \lambda} l(D, \vec{w}, w_0, \lambda)$$
$$= \arg \min_{\vec{w}, w_0, \lambda} -\frac{1}{n} \sum_{i=1}^n \left( w^T x^{(i)} + w_0 \right) y^{(i)} + \lambda (\|\vec{w}\| - 1)$$

### Gradient Update Rule With Lagrange Multiplier

$$\vec{w}^{t+1} = \vec{w}^t - \eta \left[ -\sum_{i=1}^n x_i \cdot y_i + \lambda \frac{\vec{w}}{\|w\|} \right]$$

$$w_0^{t+1} = w_0^t - \eta \left[ -\sum_{i=1}^n y_i \right]$$

Where,

- $\lambda$  - is the regularizer.